Connecting to MongoDB

MovieMatch — Team Guide

Our user accounts live in a **MongoDB Atlas** cloud database. This guide covers:

1. **Browsing the data in Compass** — quick, visual, no code
2. **Setting up Java** — driver + config file
3. **How the Java API works** — the part you actually write code against
4. **Our data shape**
5. **Troubleshooting** — read this if you get an SSL error

> **You need the connection string (URI) from Enzo.** It contains a password, so it is never committed to git. Ask for it directly.

1. Browsing the data (Compass)

Compass is a GUI for looking at the database. Useful for checking your code actually wrote what you expected. You don't need it to build features, but it helps.

1. Download **MongoDB Compass**: https://www.mongodb.com/try/download/compass
2. Open it and click **New Connection**
3. Paste the URI into the connection box:

   ```
   mongodb+srv://<username>:<password>@moviematch-users.umer8j3.mongodb.net/
   ```

   Replace `<username>` / `<password>` with the real credentials (no `<` `>` brackets).
4. Click **Connect**
5. In the left sidebar open database `moviematch` → collection `users`

From here you can view, edit, and insert documents. To add a test user, click **ADD DATA** → **Insert Document**, switch to the `{}` (JSON) view, and paste one of the examples from section 4.

Alternative if Compass won't connect: the Atlas website (cloud.mongodb.com → your cluster → **Browse Collections**) does the same thing in the browser, and works on networks that block Compass. See Troubleshooting.

2. Setting up Java

Step 1 — Add the driver to `pom.xml`

Our `pom.xml` has no dependencies yet. Add a `<dependencies>` block inside `<project>`:

```
<dependencies>
    <dependency>
        <groupId>org.mongodb</groupId>
        <artifactId>mongodb-driver-sync</artifactId>
        <version>5.2.0</version>
    </dependency>
</dependencies>
```

This one dependency pulls in `mongodb-driver-core` and `bson` automatically. After saving, let IntelliJ reload Maven (elephant icon in the Maven panel, or right-click `pom.xml` → **Maven** → **Reload project**).

Step 2 — Create your `mongo.properties`

Create a file called `mongo.properties` in the project root (same folder as `pom.xml`):

```
uri=mongodb+srv://USERNAME:PASSWORD@moviematch-users.umer8j3.mongodb.net/
database=moviematch
collection=users
```

Each person makes their own copy. This file is listed in `.gitignore`, so it will never be committed — that's deliberate, it holds a password. Do **not** paste the URI directly into a `.java` file, or it *will* end up on GitHub.

Step 3 — Check it works

If `MongoUserDataAccessObject` exists in the project, this prints the number of users:

```
MongoUserDataAccessObject dao = new MongoUserDataAccessObject("mongo.properties");
System.out.println(dao.get("bob"));   // null if there is no user named bob
dao.close();
```

If you get an `SSLException`, jump to Troubleshooting — it's your network, not your code.

3. How the Java API works

The four objects you need to know

Object	What it is	Analogy
`MongoClient`	The live connection to Atlas	The database connection
`MongoDatabase`	One database (`moviematch`)	The schema
`MongoCollection<Document>`	One collection (`users`)	A table
`Document`	One record	A row / a `Map<String, Object>`

Getting from one to the next:

```
MongoClient client = MongoClients.create(uri);
MongoDatabase database = client.getDatabase("moviematch");
MongoCollection<Document> users = database.getCollection("users");
```

> **Important:** `MongoClient` is expensive to create and is thread-safe. Create **one** for the whole app (in the DAO's constructor) and reuse it. Do not open a new one per query. Call `client.close()` once, when the app shuts down.

A `Document` is basically a Map

There is no automatic mapping to our entities — you read fields out by name:

```
Document doc = ...;
String username = doc.getString("username");
String password = doc.getString("password");
```

Field names are **case-sensitive**: `answer` and `Answer` are different fields. If you typo one, you get `null` back rather than an error, which makes for annoying bugs.

Building a document to insert:

```
Document doc = new Document("username", "bob")
        .append("displayName", "Bob")
        .append("password", "oldpass")
        .append("securityQuestion", "What is your pet's name?")
        .append("answer", "Fido");
```

The operations you'll use

Find one — returns `null` if nothing matches:

```
Document doc = users.find(Filters.eq("username", "bob")).first();
```

Check something exists:

```
boolean exists = users.find(Filters.eq("username", "bob")).first() != null;
```

Find many — iterate the result:

```
for (Document d : users.find(Filters.eq("displayName", "Bob"))) {
    System.out.println(d.getString("username"));
}
```

Insert:

```
users.insertOne(doc);
```

Update one field of one document:

```
users.updateOne(Filters.eq("username", "bob"), Updates.set("password", "newpass"));
```

Delete:

```
users.deleteOne(Filters.eq("username", "bob"));
```

Count:

```
long total = users.countDocuments();
```

`Filters` and `Updates` are helper classes — import them:

```
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;
```

Other filters you may want: `Filters.and(...)`, `Filters.or(...)`, `Filters.in(field, values)`, `Filters.gte(field, value)`. Other updates: `Updates.push(...)` (add to an array — useful for watchlists), `Updates.pull(...)` (remove from an array).

Where this fits in Clean Architecture

The rule: **only the DAO knows MongoDB exists.** Interactors depend on a data-access *interface* in the `use_case` package; the Mongo class implements it. That means you can swap in an in-memory fake for testing without changing any business logic.

```
use_case/…/SomethingUserDataAccessInterface   ← interactor depends on this
              ▲ implements
data_access/MongoUserDataAccessObject         ← only this file imports com.mongodb
```

So: **never** import `com.mongodb` inside `use_case/`, `interface_adapter/`, `entity/`, or `view/`.

Full working example

This DAO implements the interfaces for the security-question and reset-password use cases. It compiles against our project as-is — use it as the template for your own DAO methods.

```
package data_access;

import java.io.FileInputStream;
import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

import org.bson.Document;

import com.mongodb.client.MongoClient;
import com.mongodb.client.MongoClients;
import com.mongodb.client.MongoCollection;
import com.mongodb.client.MongoDatabase;
import com.mongodb.client.model.Filters;
import com.mongodb.client.model.Updates;

import entity.StandardUser;
import entity.User;
import use_case.reset_password.ResetPasswordUserDataAccessInterface;
import use_case.security_question.SecurityQuestionUserDataAccessInterface;

/**
 * MongoDB-backed data access object for users.
 */
public class MongoUserDataAccessObject
        implements SecurityQuestionUserDataAccessInterface, ResetPasswordUserDataAccessInterface {

    // Field names as stored in MongoDB. These must match the documents exactly.
    private static final String USERNAME = "username";
    private static final String DISPLAY_NAME = "displayName";
    private static final String PASSWORD = "password";
    private static final String SECURITY_QUESTION = "securityQuestion";
    private static final String ANSWER = "answer";

    private final MongoClient mongoClient;
    private final MongoCollection<Document> users;

    /**
     * Opens a connection using the settings in mongo.properties.
     * @param propertiesPath path to the properties file (e.g. "mongo.properties")
     */
    public MongoUserDataAccessObject(String propertiesPath) {
        final Properties props = new Properties();
        try (InputStream in = new FileInputStream(propertiesPath)) {
            props.load(in);
```

```
        }
        catch (IOException ex) {
            throw new RuntimeException("Could not read " + propertiesPath, ex);
        }

        this.mongoClient = MongoClients.create(props.getProperty("uri"));
        final MongoDatabase database = mongoClient.getDatabase(props.getProperty("database"));
        this.users = database.getCollection(props.getProperty("collection"));
    }

    @Override
    public boolean existsByName(String username) {
        return users.find(Filters.eq(USERNAME, username)).first() != null;
    }

    @Override
    public User get(String username) {
        final Document doc = users.find(Filters.eq(USERNAME, username)).first();
        if (doc == null) {
            return null;
        }
        return toUser(doc);
    }

    @Override
    public void changePassword(String username, String newPassword) {
        users.updateOne(Filters.eq(USERNAME, username), Updates.set(PASSWORD, newPassword));
    }

    /**
     * Saves a brand-new user document.
     * @param user the user to insert
     */
    public void save(User user) {
        final Document doc = new Document(USERNAME, user.getUsername())
                .append(DISPLAY_NAME, user.getDisplayName())
                .append(PASSWORD, user.getPassword())
                .append(SECURITY_QUESTION, user.getSecurityQuestion())
                .append(ANSWER, user.getSecurityAnswer());
        users.insertOne(doc);
    }

    /** Converts a MongoDB document into a User entity. */
    private User toUser(Document doc) {
        return new StandardUser(
```

```
                doc.getString(USERNAME),
                doc.getString(DISPLAY_NAME),
                doc.getString(PASSWORD),
                doc.getString(SECURITY_QUESTION),
                doc.getString(ANSWER));
    }

    /** Closes the connection. Call once when the app shuts down. */
    public void close() {
        mongoClient.close();
    }
}
```

Adding your own use case's DAO methods

Don't write a second `MongoClient`. Instead:

1. Define what you need in your use case's data-access interface, e.g. `use_case/watchlist/WatchlistUserDataAccessInterface`.
2. Add `implements WatchlistUserDataAccessInterface` to `MongoUserDataAccessObject` and implement the methods there, reusing the same `users` collection.
3. Pass the same DAO instance into your interactor in `AppBuilder`.

4. Our data shape

Database `moviematch`, collection `users`. One document per user:

```
{
  "username": "bob",
  "displayName": "Bob",
  "password": "oldpass",
  "securityQuestion": "What is your pet's name?",
  "answer": "Fido"
}
```

A second one for testing:

```
{
  "username": "alice",
  "displayName": "Alice",
  "password": "hunter2",
  "securityQuestion": "Which city were you born in?",
  "answer": "Toronto"
}
```

These map to `entity.StandardUser`:

Mongo field	`User` method
`username`	`getUsername()`
`displayName`	`getDisplayName()`
`password`	`getPassword()`
`securityQuestion`	`getSecurityQuestion()`
`answer`	`getSecurityAnswer()`

Mongo also adds an `_id` field automatically — ignore it, we key on `username`.

> **Note:** passwords and security answers are stored as plain text. That's fine for this project, but it is not what you'd do in production — you'd store a hash instead.

5. Troubleshooting

`SSLException: Received fatal alert: internal_error`

This is the most common one, and **it is not a bug in your code**. The connection is being blocked or intercepted before it completes. Causes, in order of likelihood:

1. **You're on a school/campus network or VPN.** UofT wifi and many VPNs block or inspect traffic on MongoDB's port (27017). **Fix: use a phone hotspot**, or disconnect the VPN. This solves it most of the time.
2. **Antivirus doing HTTPS/SSL scanning** (Avast, AVG, Kaspersky, ESET, Bitdefender). Turn off the "web shield" / "encrypted connection scanning" option.
3. **Outdated Compass** — update to the latest version.

Note the Atlas *website* still works on blocked networks (it uses normal port 443), so you can always add/inspect data at cloud.mongodb.com → **Browse Collections** even when Compass and Java can't connect.

`MongoTimeoutException` / connection just hangs

- **Your IP isn't allowed.** Atlas → **Network Access** → **Add IP Address** → *Allow access from anywhere* (`0.0.0.0/0`). Wait ~1 minute for it to become Active.
- **The cluster is paused.** Free-tier clusters pause after inactivity — Atlas will show a **Resume** button.

`MongoSecurityException` / authentication failed

- Wrong username or password in the URI.
- You left the `<` `>` brackets in — `<password>` must be replaced entirely.
- Your password contains special characters. `@ : / ? # %` break the URI and must be percent-encoded (`@` → `%40`, `#` → `%23`). Easiest fix: use a password with only letters and numbers.

Code runs but every field is `null`

Your field names don't match the documents. Check spelling and capitalisation against section 4 — `answer` is not `securityAnswer`, `displayName` is not `displayname`.

`SLF4J not found on the classpath` warning

Harmless. It just means logging is disabled. Ignore it, or add an SLF4J dependency if the noise bothers you.